

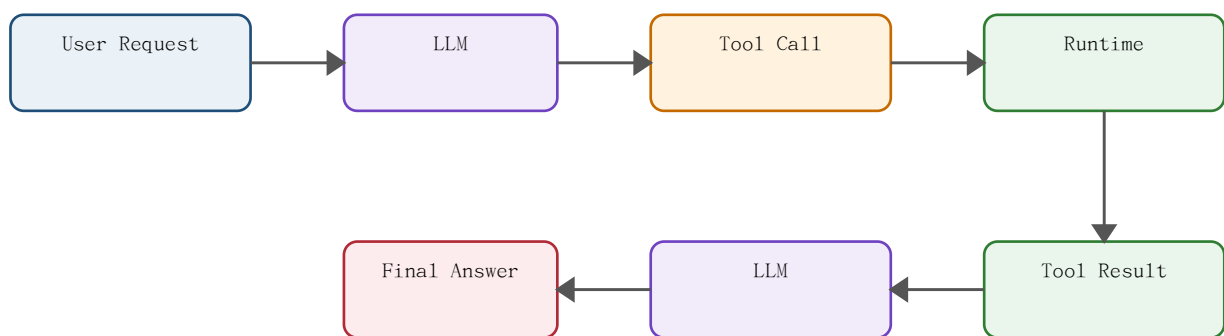
第27讲：Tool Use / Function Calling

大语言模型怎样从“会说话”变成“会调用外部工具”？

本讲目标

学完后你应该能解释：Function Calling 为什么本质上仍然是 token / 结构化输出；模型、Tool Schema、Runtime、Validator

和真实工具分别负责什么；为什么“模型选择调用函数”不等于“模型已经执行函数”；以及 Agent Tool Use 为什么会带来新的可靠性与安全问题。



模型负责“提出结构化动作”；Runtime 负责“真正执行动作”。

先记住一句：LLM 通常只负责生成“动作意图”，真正改变外部世界的是 Runtime 和 Tool。

1. 为什么纯 LLM 不够？

纯语言模型最擅长的是根据 Context 继续生成 token。但很多真实任务不是“知道答案”就结束，而是必须访问外部状态或执行动作。

任务	只靠 LLM 的问题	需要的工具
今天的天气	参数知识可能过时	Weather API
精确计算	可能算错	Calculator / Python
读本地文件	模型本身看不到文件系统	File Tool
查数据库	模型不能凭空知道实时记录	Database Query
发邮件	生成文字 ≠ 真正发送	Email Tool
创建日程	回答“已安排”不等于真的写入日历	Calendar Tool

核心变化

Tool Use

让模型从“只生成语言”扩展到“生成可执行动作描述”，再由外部系统把动作落到现实环境。

2. Function Calling 的本质仍然是“生成 token”

假设系统提供一个工具：

```
{ "name": "get_weather", "description": "Get weather for a city", "parameters": { "city": "string", "date": "string" } }
```

用户问：“明天北京天气怎么样？”模型可能生成的不是自然语言答案，而是一段结构化 Tool Call：

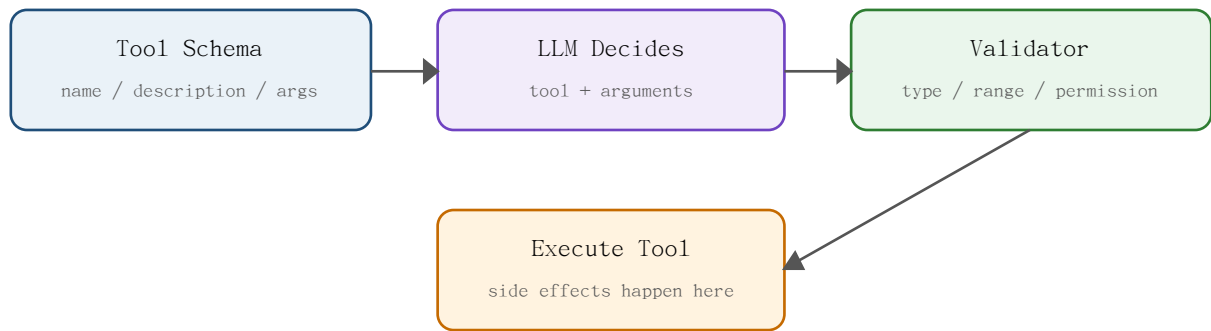
```
{ "tool": "get_weather", "arguments": { "city": "Beijing", "date": "tomorrow" } }
```

从 LLM 内部看，这仍然是下一 token 预测，只是训练和解码约束让输出遵循某种结构。

非常重要

模型输出 JSON / function call，并不表示天气 API 已经被调用。它只是生成了一个“请 Runtime 调这个工具”的结构化请求。

3. Tool Schema：模型怎样知道有哪些工具？



Schema 是接口契约；Validator 是边界；Execution 才真正改变外部世界。

Tool Schema 就像给模型一份“接口说明书”。它通常包含：

字段	作用
name	工具叫什么
description	什么情况下应该用
parameters	有哪些参数、类型是什么
required	哪些参数必填
constraints	允许值、范围、枚举等

Schema 的描述质量会直接影响 Tool Selection 和 Argument Generation。描述含糊、多个工具高度重叠，都可能导致调用错误。

4. Runtime: Tool Use 真正的“执行层”

Runtime 接到模型生成的 Tool Call 后，至少要做：

Parse -> Validate -> Authorize -> Execute -> Capture Result -> Return to LLM	
步骤	问题
Parse	输出是否是合法结构？
Validate	参数类型、格式、范围对不对？
Authorize	当前用户/Agent 有权限吗？
Execute	真正调用 API / 文件 / 数据库 / 程序
Capture	记录返回值、错误码、日志
Return	把 Tool Result 重新放进模型 Context

底层边界

LLM 是 probabilistic policy; Runtime 才是 deterministic boundary。不要把安全责任全部交给语言模型“自觉”。

5. 一个完整 Tool Loop 到底怎样跑？

以“计算 173×294 ”为例：

```
User: 计算 173 * 294 | LLM: {"tool":"calculator","arguments":{"a":173,"b":294,"op":"mul"}} |  
Runtime: validate -> execute | Tool Result: {"value":50862} | LLM: 173 * 294 = 50862
```

注意：最终自然语言答案通常是在 Tool Result 回填以后，模型又做了一次 inference 才生成。

所以一次看起来很简单的工具调用，背后可能至少包含：

LLM inference #1 -> Tool execution -> LLM inference #2

6. Tool Call vs Tool Result: 不要混

Tool Call	Tool Result
模型提出“我要做什么”	外部系统告诉模型“实际发生了什么”
可能是错的	也可能失败/超时/返回空值
包含工具名与参数	包含返回值/错误码/状态
尚未改变外部世界	执行完成后外部状态可能已改变

常见事故

模型说“文件已删除”并不代表文件真的被删除。必须以 Tool Result / Runtime 状态为准，而不是以模型生成的叙述为准。

7. Tool Selection: 模型什么时候应该调用工具？

模型并不是所有问题都要调用工具。一般要判断：

情况	更合理的策略
常识性解释	直接回答
实时信息	调用外部查询
精确算术	调用计算器
用户私有数据	授权后调用对应数据源
需要改变外部状态	调用有副作用的工具
证据不足	先检索/查询，而不是猜

这实际上是一个 Policy Learning

问题：模型要学习“什么时候调用、调用哪个、参数是什么、什么时候停止”。

8. Argument Generation: 最常见的失败点之一

即使工具选对了，参数仍可能出错。例如：

```
[[{"tool": "book_table", "arguments": {"party_size": 4, "time": "19:00"}}] [{"tool": "book_table", "arguments": {"party_size": "many", "time": "evening"}}]
```

因此 Runtime 必须做 Schema Validation，而不是相信模型输出。

参数错误类型	例子
类型错误	数字参数输出字符串
缺失参数	忘记 required field
格式错误	日期/时间格式不合法
范围错误	party_size = -3
语义错误	把 destination 写进 origin
幻觉参数	Schema 根本没有这个字段

9. Structured Output: 为什么 JSON 很重要？

如果模型只用自然语言说：

```
“我建议调用天气工具，城市是北京。”
```

Runtime

还要自己猜测这句话怎样映射到函数参数。结构化输出可以把语言意图变成机器可验证的数据结构。

```
[[{"tool": "weather", "arguments": {"city": "Beijing"}}]] -> [[{"tool": "weather", "arguments": {"city": "Beijing"}}]] Action -> Validator -> Tool
```

注意

JSON 只是常见表示。真正重要的是“结构化 + 可校验”，并不是 JSON 这个格式本身。

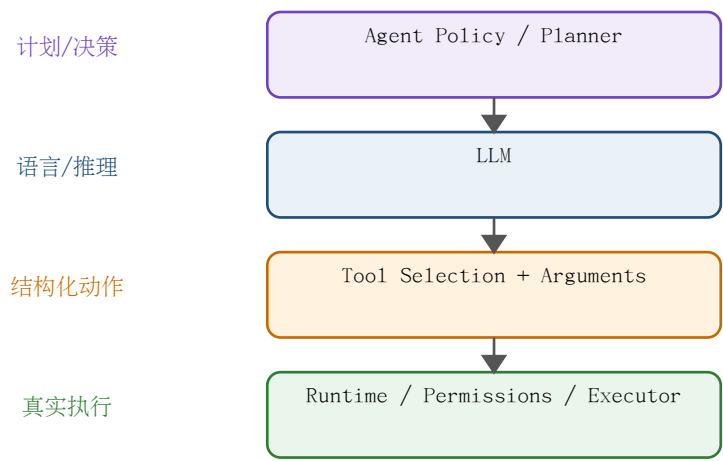
10. Tool Error: 工具会失败，Agent 必须会处理失败

真实工具可能超时、限流、权限不足、返回 404、格式变化、部分成功。Agent 不能把“调用了工具”当成“任务已经完成”。

Tool Result	Agent 应怎样处理
success	解析结果，继续下一步
timeout	重试/换工具/报告失败
rate limit	等待或 backoff
permission denied	请求授权，不得绕过
not found	检查路径/查询参数
partial result	标记不完整，不要伪装完整

最低可靠性原则
所有 side effect 都应该以外部返回状态为准。Agent 的“自述成功”不能作为成功证据。

11. Tool Use 如何把 LLM 变成 Agent 的一部分？



Tool Use 把“文本生成”接到“外部状态变化”上。

单次 Function Calling 还不等于完整 Agent。Agent 通常还需要：

Goal / State / Planning / Memory / Tools / Environment Feedback / Loop

Tool Use 解决的是其中一个关键接口：让语言模型能够通过 Runtime 对环境采取行动并读取反馈。

12. Tool Use 与 RAG 的区别

RAG	Tool Use
主要获取信息	既能获取信息，也能执行动作
常见结果是文本证据	结果可以是数据、状态、side effect
通常不会改变外部世界	工具可能发送、删除、购买、修改
核心问题是 retrieval quality	还要考虑权限、参数、安全与执行

实际上，RAG 也可以被看成一种特殊工具链：Retriever 是工具，返回 evidence，再交给 LLM。

13. Tool Hallucination: 模型会“幻想工具存在”吗？

会。模型可能生成一个 Schema 中不存在的工具名，或者给已有工具发不存在的参数。

例如系统只有：

```
search_docs(query)
```

模型却生成：

```
search_web_and_download_pdf(url, save_to)
```

这就是 Tool Hallucination / Invalid Tool Call。Runtime 必须拒绝，而不是偷偷猜一个最接近工具。

科研指标

Tool Selection Accuracy、Argument Accuracy、Invalid Call Rate、Execution Success Rate、Task Success Rate 应该分开测。

14. Tool Use 的安全边界：读和写不是同一种风险

工具类别	风险示例	建议
Read-only	读文件、搜索、查询天气	相对低风险，但仍有隐私问题
Write	写文件、改数据库	需要验证目标与范围
External Communication	发邮件、发消息	通常需要明确授权/预览
Financial / Purchase	购买、付款	高风险，需强权限与确认
Destructive	删除、覆盖、终止服务	最小权限 + 明确确认 + 审计

系统原则

权限应该由 Runtime / OS / API token 强制执行，而不是只靠 Prompt
写一句“请不要做危险操作”。

15. Least Privilege: Agent 为什么应该只拿最小权限？

如果一个只需要读取日志的 Agent 却拿到了删除数据库权限，那么一次错误 Tool Call 就可能造成不必要的高后果。

所以要遵循：

```
task requirement -> minimal tool set -> minimal scope -> minimal duration
```

例如只允许读取某个项目目录，而不是整个文件系统；只允许生成邮件草稿，而不是直接发送。

16. Confirmation: 什么时候需要人工确认？

动作	通常是否需要确认
读取公开网页	通常不需要
运行无副作用计算	通常不需要
修改本地文件	视影响范围
发送外部消息	常需要
删除重要数据	应需要
付款/下单	应需要强确认

是否确认不是模型“感觉危险不危险”，而应该由 Tool Policy 根据动作类别和上下文决定。

17. Tool Result 也可能被恶意操纵

如果工具返回的是网页、文档、第三方 API 文本，这些结果可能包含恶意指令。模型可能把“数据”误当成“更高优先级指令”。

这和第26讲 RAG Prompt Injection 是同一类问题：

untrusted external text -> context -> model policy may be hijacked

防护思路
明确标记 tool result
为数据；限制工具权限；不要允许外部文本直接决定高风险动作；对关键动作使用独立 validation / confirmation。

18. Tool Feedback: 工具是一个“外部真值通道”

工具不仅能执行动作，还能给模型提供比语言先验更可靠的反馈。例如：

问题	外部验证
代码是否运行成功	exit code / tests
文件是否存在	filesystem result
SQL 是否成功	database response
网页是否可访问	HTTP status

问题	外部验证
计算是否正确	calculator result

这就是 Agent 很重要的一种闭环:

Reason -> Act -> Observe -> Update -> Reason again

19. Agent Tool Use 的错误传播

一次错误 Tool Call 可能改变环境状态，然后后续所有推理都基于这个错误状态继续。

Wrong Tool -> Wrong Side Effect -> Wrong Observation -> Wrong Memory -> More Wrong Actions

AI Systems Science 视角
Tool Use 把 LLM 的错误从“文本错误”升级成“状态转移错误”。研究时应该测错误是否被检测、是否可回滚、是否跨步骤传播。

20. 多 Agent + Tools: 资源和权限问题会放大

多个 Agent 如果都可以调用工具，会出现新的系统变量:

变量	研究问题
Tool Budget	多个 Agent 总调用数是否公平?
Tool Contention	多个 Agent 是否争用同一资源?
Permission Heterogeneity	不同 Agent 是否有不同权限?
Tool Specialization	不同 Agent 是否拥有不同工具?
Shared State	一个 Agent 的操作是否影响其他 Agent?
Audit Trail	能否追踪是谁做了哪一步?

与你的研究连接
Tool Set 的长期差异可能形成稳定功能异质性，但必须控制总调用预算，并区分“工具更多”与“协作更聪明”。

21. Tool Use 怎么评测？不要只看最终任务成功率

指标	测什么
Tool Selection Accuracy	是否选对工具
Argument Accuracy	参数是否正确
Invalid Call Rate	是否调用不存在工具/非法参数
Execution Success Rate	Runtime 执行是否成功
Recovery Rate	工具失败后能否正确恢复
Task Success Rate	最终任务是否完成
Side-effect Error Rate	是否造成不该发生的外部修改
Calls / Tokens / Cost	系统资源开销

实验原则

Tool Use 论文如果只报告 Task Success，很难知道提升来自更会选工具、更会填参数，还是只是多调用了几次。

22. 小实验：模型提议，Runtime 执行

材料包里的 Python 实验实现一个极小 Tool Runtime，包含三个工具：

```
calculator(a, b, op) lookup_record(key) file_exists(path)
```

实验会演示：

场景	现象
合法调用	通过 schema validation 后执行
不存在工具	Runtime 拒绝
参数缺失	Runtime 拒绝
危险路径	权限策略拒绝
Tool Result 回填	模型拿真实结果生成最终答案

关键

Toy model 只是模拟“LLM 生成 Tool Call”。真正值得观察的是 Runtime：任何模型输出都必须先经过解析、校验和权限检查，再执行。

23. 常见误区汇总

误区	更准确的理解
模型生成 function call = 函数已执行	只是动作请求，Runtime 才执行
工具返回成功，模型一定理解正确	模型仍可能误读 Tool Result
Prompt 里写安全规则就够了	真正权限要由 Runtime 强制

误区	更准确的理解
Agent 会调用工具 = Agent 很智能	可能只是模板化 tool routing
工具越多越强	工具选择难度和攻击面也更大
调用次数越多 = 推理越充分	也可能只是浪费预算或错误循环
多 Agent 工具使用更强	必须控制总 tool calls / tokens / side effects

24. 练习题

- 1) 为什么 Function Calling 本质上仍然可以理解为 token 生成？
- 2) Tool Call 和 Tool Result 最根本的区别是什么？
- 3) 为什么 Runtime 必须做 Schema Validation？
- 4) 为什么“模型说文件已删除”不能作为文件真的删除的证据？
- 5) Tool Selection 与 Argument Generation 应该怎样分开评测？
- 6) 什么叫 Least Privilege？为什么 Agent 特别需要它？
- 7) 为什么 Tool Result 本身也可能成为 Prompt Injection 来源？
- 8) RAG 和 Tool Use 有什么关系与区别？
- 9) 多 Agent 共用同一组工具时，为什么要控制 Tool Budget？
- 10) 设计一个最小实验，区分“模型更会调用工具”和“只是多调用工具”。

答案见材料包《03_第27讲_练习题与答案.pdf》。

25. 本讲总结

一句话总纲 Tool Use 的本质，是让 LLM 把语言推理转化成结构化动作请求，再由受权限约束的 Runtime 真正执行，并把外部结果重新送回模型。模型负责“提议”，系统负责“验证与执行”。	
组件	最短记法
Tool Schema	告诉模型工具长什么样
LLM	选择工具并生成参数
Validator	检查结构、类型、范围
Permission Layer	判断是否允许
Executor	真正调用外部系统
Tool Result	外部世界返回的事实
Agent Loop	基于结果继续规划/行动
下一讲：第28讲《从 LLM 到 Agent: Planning、State、Memory、Tools、Environment 与 Action Loop》。	